

Reinforcement Learning: From Foundations to Deep Approaches

Summer Semester 2024, Homework 3

Prof. Georgia Chalvatzaki, Dr. Davide Tateo

Total points: 103

Deadline: 10:00, Friday 25th July 2025



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Use the provided Python scripts to solve the problems. We provided you with a zip file for the homework project that includes the Python scripts (to complete) and bash scripts (to run experiments). The structure of the project is similar to MushroomRL.

We encourage you to use Conda to create an environment that contains all packages necessary for solving the homework. We provide you with a yaml file (environment.yml) that you can use to set up the conda environment as follows: "conda env create -file=environment.yml". This assignment will not be graded! The solutions will be discussed on Friday 25th July 2025 at 10:00 am, during the practical session.

3.1 Implementation [30 Points]

For this project, you will be part of the team responsible for giving feedback to the athletes of your country who are participating in the Paris Olympic Games this summer. In order to improve the athlete's gait for the 100-meter hurdles race, you have the idea of leveraging the knowledge you learned during the Reinforcement Learning (RL) lecture. You will train an RL agent, Hopper, which learns how to run while having to jump over two hurdles (see Figure 1). To solve this reinforcement learning challenge, we will use three algorithms: Proximal Policy Optimization (PPO) [1], Twin Delayed Deep Deterministic Policy Gradient (TD3) [2], and Soft Actor-Critic (SAC) [3].

For this question, you will implement the environment interface and set up the learning agents and networks. The code in the Python files requiring your attention will be annotated with '# TODO'. Complete these to implement the training process using the specified reinforcement learning algorithms. You ask you to only insert lines of code between the comments '# [START YOUR CODE HERE]' and '# [END YOUR CODE HERE]'.

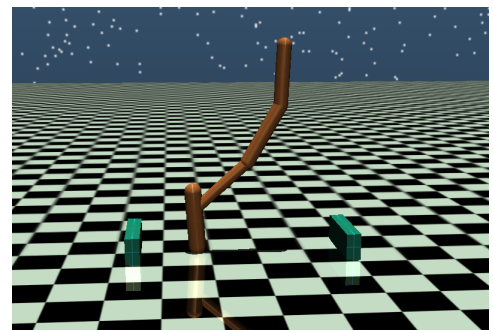


Fig. 1: Hopper agent in the simulated Olympic hurdles environment with Hopper (brown) and hurdles (green).

1. The mdp can be found in *mushroom_rl_extensions/environments/mujoco_envs/data/hopper_fixed_hurdles.xml*. You must complete the gaps in *mushroom_rl_extensions/environments/mujoco_envs/hopper_hurdles.py* to complete the mdp interface. Details on the implementation are provided in the python file. [5 Points]

Solution:

Give full points if the provided code produces a similar output as:

```
healthy_reward = self._get_healthy_reward(next_obs)
forward_reward = self._get_forward_reward(self._forward_reward_weight)
ctrl_cost = self._get_ctrl_cost(action, self._ctrl_cost_weight)
```

2. The algorithms you will train are constructed in *setup_algorithms.py*. Fill in the gaps to construct the networks in each algorithm appropriately. [10 Points]

Solution:

Give full points if the provided code produces a similar output as:

```
For PPO:
critic_input_shape = mdp.info.observation_space.shape
For TD3:
actor_input_shape = mdp.info.observation_space.shape
actor_output_shape = mdp.info.action_space.shape
critic_input_shape = mdp.info.observation_space.shape[0] + mdp.info.action_space.shape[0]
```

3. The networks you will use for the agents are described in *networks.py*. Fill in the gaps in this file to construct the networks according to the architectures described. [10 Points]

Solution:

Give full points if the provided code produces a similar output as:

```
For ActionValueCriticNetwork:
self._h1 = nn.Linear(n_input, n_features)
self._h2 = nn.Linear(n_features, n_features)
self._h3 = nn.Linear(n_features, n_output)
nn.init.xavier_uniform_(self._h1.weight, gain=nn.init.calculate_gain("relu"))
nn.init.xavier_uniform_(self._h2.weight, gain=nn.init.calculate_gain("relu"))
nn.init.xavier_uniform_(self._h3.weight, gain=nn.init.calculate_gain("linear"))
For ValueCriticNetwork:
features1 = F.relu(self._h1(torch.squeeze(state, 1).float()))
```

4. The scripts for on-policy and off-policy training are detailed in the *run_single_experiment.py*. Fill in the gaps in this file to enable the on-policy and off-policy training scripts to function correctly. [5 Points]

Solution:

Give full points if the provided code produces a similar output as:

```
core.learn( n_steps=n_steps_training_per_epoch, n_steps_per_fit=n_steps_per_fit, quiet=True, render=False, )
```

3.2 Tuning [45 Points]

Hyperparameter tuning is a critical aspect of solving high-dimensional problems in deep reinforcement learning systems. For this exercise, you will adjust specific algorithmic hyperparameters. Ensure that the hyperparameters stay within the specified ranges during each experiment. To speed up the hyperparameter tuning, we allow you to tune the hyperparameters for only one seed. However, note that in practice, multiple seeds are required to have significant results.

We have prepared a simple tool for tuning the hyperparameters. All hyperparameters are stored in YAML files under the *configs* directory. Adjust the hyperparameters to optimize the agent's performance. The results will be recorded in the folder under the *out/{hash}*, where *hash* is an 8-character hash code generated from the corresponding parameters to ensure uniqueness. The purpose of this assignment is to provide a more intuitive understanding of the various impacts of different hyperparameters in the algorithms.

For debugging purposes, you can execute the command `python watch_agent.py -ckpt {path to best-agent.msh}` on your local machine to visualize and assess the agent's performance.

For visualization, you can execute the command `python logger_visualizer.py -p {logger directory}` to generate the plots. This will allow you to monitor the changes in R , J , V , and H .

Store the best agents of SAC, TD3, and PPO algorithms in a folder named "best agents". This folder should include the *logger.log* file, the *config.yml* file and the *agent-best.msh* file, with each subfolder named according to the algorithm. For example, the files for the SAC algorithm should be stored in *best_agents/sac*.

Generate and add the plots for R , J , V , and H for the best-performing agent of each algorithm in the report. You can achieve the optimal solution by using different seeds and combinations of parameters. It is best not to alter other hyperparameters; however, you are entirely capable of conducting a comprehensive analysis of each hyperparameter, as the config file allows for this. Nonetheless, we recommend starting with the range we suggest. We provide a logger visualizer to export the desired plots in *logger_visualizer.py*.

1. **Tune the following algorithm-specific hyperparameters:** (a) the learning rate η of SAC within the range $[0.0005, 0.001]$, (b) the policy sigma σ of TD3 within the range $[0.0001, 0.01]$, and (c) the clipping threshold ϵ for PPO within the range $[0.1, 0.3]$. **Modify the respective YAML files in the configs directory for the above adjustments. The results of the runs will be stored in the following directory structure: *out/logs/hopper_fixed_hurdles/{ppo/sac/td3}/{seed}* [45 Points]**

Solution:

Each subquestion (a), (b), and (c) is worth 15 points. The reported plots should be close to the following plots. Several cases can occur:

- Give 15 points if the plot looks similar to the one of the solution.
- Give 10 points if the plot looks worse than the one of the solution, but R reaches 900.
- Give 0 points otherwise.

3.3 Analysis of SAC's agent [25 Points]

In this exercise, we will investigate different metrics related to the trained SAC agent. Execute the command `python analyze_sac.py -p {logger directory}` to generate two plots that we will analyze. We ask you to add them to the report.

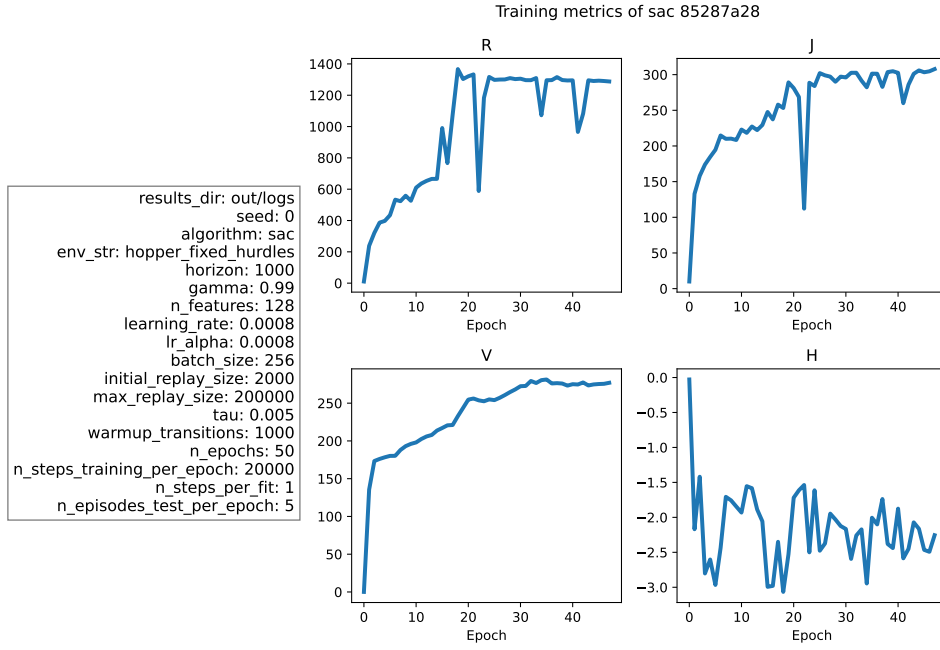


Fig. 2: Tuned SAC.

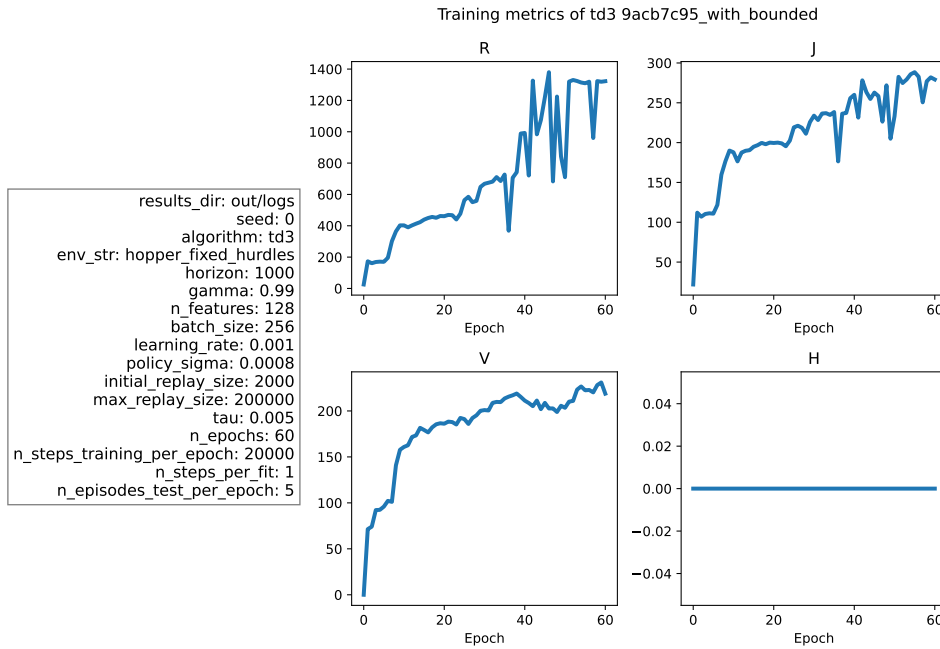


Fig. 3: Tuned TD3.

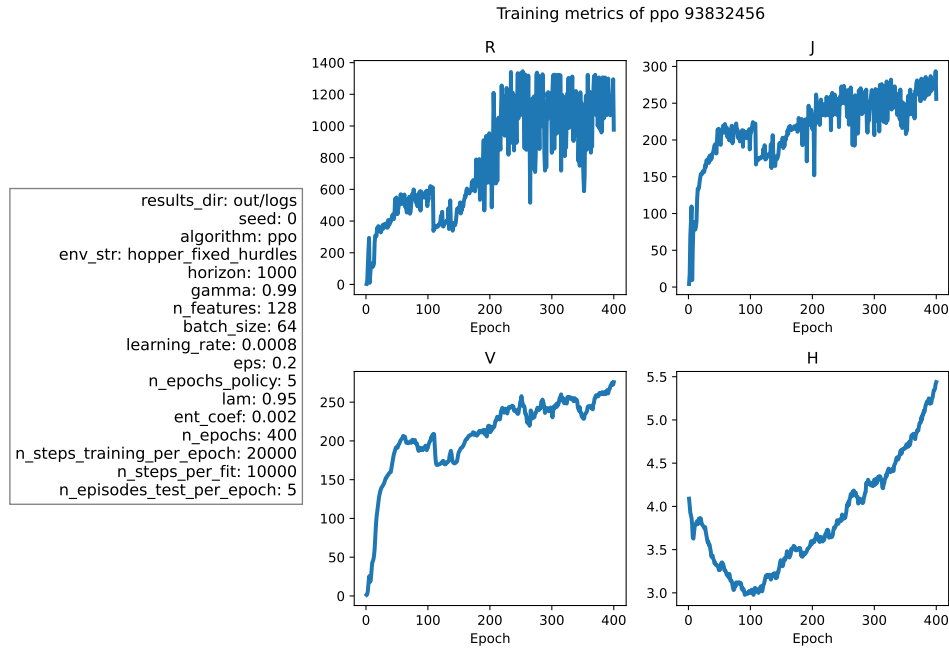


Fig. 4: Tuned PPO.

1. We start by analyzing the plot named *Analyze_SAC_physical_quantities_{hash}_plot.pdf*. (a) Using the Gymnasium documentation, describe what 'actions[:, 2]', 'state[:, 0]' and 'states[:, 11]' stand for. (b) Comment on the shape of the Q function prediction along the x-axis. For that, we advise looking at the reward function of the environment. (c) Explain why 'actions[:, 2]' is negative before the first vertical black line. [15 Points]

Solution:

The reported plot should be close to the following plot. Several cases can occur:

- Give 2 points if the student correctly identified 'actions[:, 2]' to the torque applied on the foot rotor.
 - Give 2 points if the student correctly identified 'state[:, 0]' to the z-coordinate of the torso.
 - Give 1 points if the student correctly identified 'states[:, 11]' to the x-coordinate of the torso.
 - Give 5 points if the student explained that the Q estimates are increasing because, when x is small, the discount factor makes the reward given at $x = 4.5$ small. When x grows, the state $x = 4.5$ becomes closer so the reward given at $x = 4.5$ is less discounted.
 - Give 5 points if the student explained that 'actions[:, 2]' is negative before the first vertical black line because the first vertical black line represents the first hurdle. Since 'actions[:, 2]' is the torque applied on the foot rotor, applying a negative torque to the foot right before the first hurdle would help Hopper to jump.
2. We now analyze the plot named *Analyze_SAC_Q_and_pi_{hash}_plot.pdf*. (a) Explain the values of the Q-function according to the values of the actions. (b) Has the policy been well-trained for the considered state? Two reasons supporting your answer are expected. Be careful. In some cases, the left plot might seem empty. This can happen if the policy focuses on one edge of the plot. In this case, make sure to identify the edge before making the analysis. [10 Points]

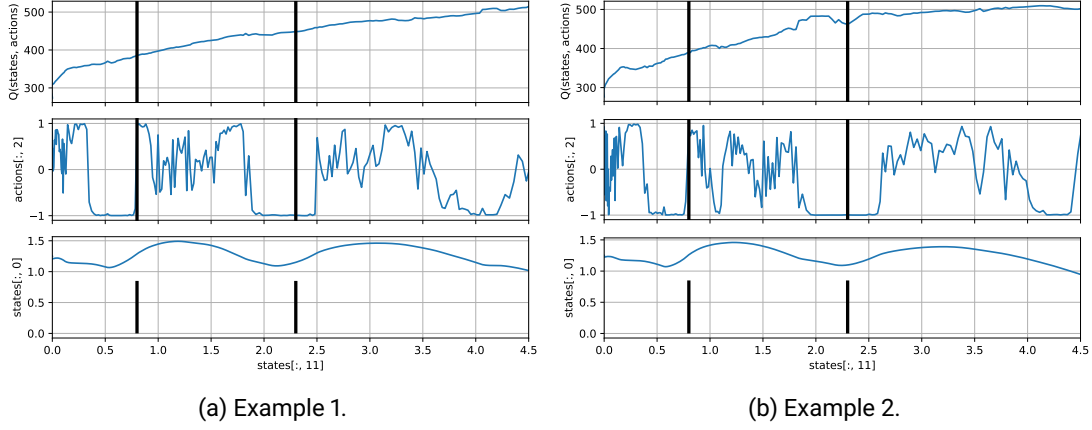


Fig. 5: Analysis of the physical quantities of SAC's agent.

SAC agent's characteristics

$E[Q(s, \pi(s))] \approx 344$, where s is a state right before the first hurdle ($x \approx 0.5$)

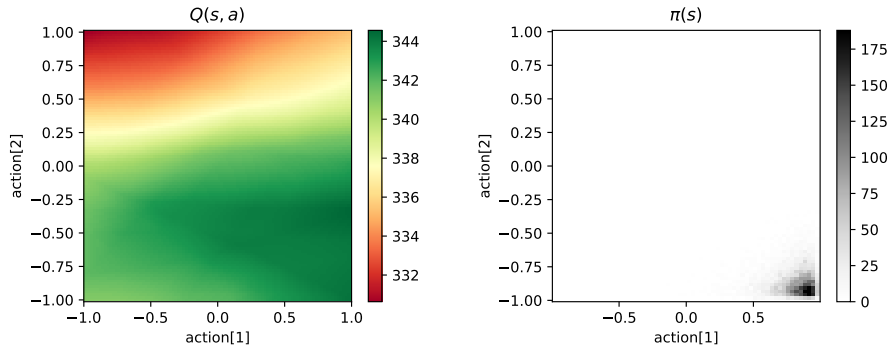


Fig. 6: Analysis of the Q and π of SAC's agent. Example 1.

Solution:

The reported plot should be close to the following plot. Several cases can occur:

- Give 2 points if the student explained that the action[1] seems to be less important than action[2]. This can be explain by the fact that action[2], i.e. the torque applied to the foot, is more important for the agent to jump than action[1], i.e. the torque applied to the leg.
- Give 3 points if the student explained that the Q estimate is higher for lower values of action[2]. This comes from the fact that the higher the torque is, the higher Hopper will jump.
- Give 3 points if the student explained that the region where the actions are sampled is the region where the Q estimate is higher.
- Give 2 points if the student explained that the expected Q estimate under the policy, i.e. $\mathbb{E}_{a \sim \pi(s)}[Q(s, a)]$, corresponds to a high value of the Q estimate when looking at the color bar of the plot showing the Q estimate.

SAC agent's characteristics

$E[Q(s, \pi(s))] \approx 333$, where s is a state right before the first hurdle ($x \approx 0.5$)

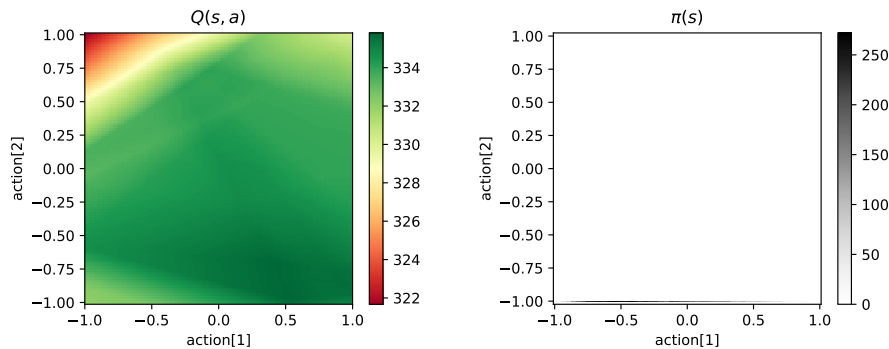


Fig. 7: Analysis of the Q and π of SAC's agent. Example 2. Caution!! The policy outputs actions with action[2] close to -1.

3.4 Bonus - TD3 without Tanh [3 Points]

1. Train a TD3 agent that does not use a Tanh activation function after the last layer for the actor. For that, you can use the class `ActorNetwork` instead of `BoundedActorNetwork`. What is the main issue affecting the performance? Report the plot showing R , J , V , and H . [3 Points]

Solution:

The reported plot should be close to the following plot. Give 3 points if the student explained that the tanh activation function prevents the noise added to the actor's output from going outside of the action space.

Notes: If the code is implemented correctly, training an agent should not take more than 2-4 hours to complete.

Reference

- [1] John Schulman et al. "Proximal policy optimization algorithms". In: *arXiv preprint arXiv:1707.06347* (2017).
- [2] Scott Fujimoto, Herke Hoof, and David Meger. "Addressing function approximation error in actor-critic methods". In: *International conference on machine learning*. PMLR. 2018, pp. 1587–1596.
- [3] Tuomas Haarnoja et al. "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor". In: *International conference on machine learning*. PMLR. 2018, pp. 1861–1870.

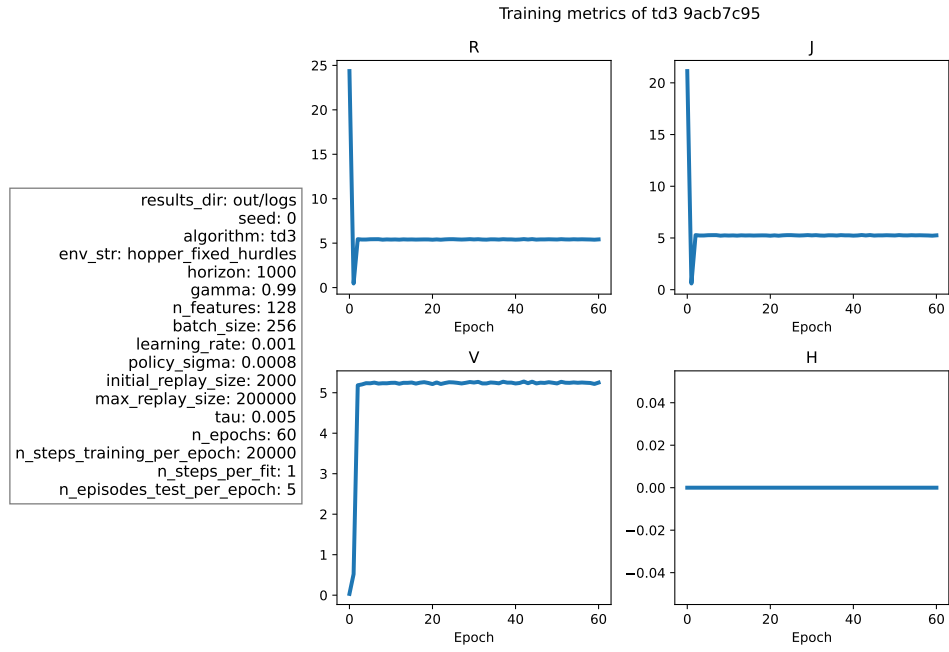


Fig. 8: TD3 trained without tanh.